ASSIGNMENT

Open Source Web Programming using PHP

(Paper 1T3)

First Year M.C.A. — Semester I (CBCS)

RTM Nagpur University

Name: _____

Roll No: _____

Q1. How to install and configure PHP in windows? Explain

PHP is a server-side scripting language, so it needs a **web server (Apache)** and usually a **database (MySQL)** to run web pages. On Windows, the easiest way is to install a package such as **XAMPP** or **WAMP**, which contains Apache, PHP and MySQL together.

Installation using XAMPP

- **Step 1:** Download XAMPP for Windows from apachefriends.org and run the installer.
- **Step 2:** Select the components **Apache, MySQL, PHP and phpMyAdmin** and choose the folder (default C:\xampp).
- **Step 3:** Open the **XAMPP Control Panel** and click **Start** for Apache and MySQL.
- **Step 4:** Test the server — open http://localhost in the browser; the XAMPP dashboard appears.
- **Step 5:** Save PHP files in **C:\xampp\htdocs** (the document root), e.g. htdocs\test.php containing <?php phpinfo(); ?>, and open http://localhost/test.php.

Configuration

- **php.ini** (C:\xampp\php\php.ini) is the main configuration file. Common settings: display_errors = On, upload_max_filesize, max_execution_time, date.timezone = Asia/Kolkata.
- **Enable extensions** by removing the semicolon, e.g. extension=mysqli, extension=gd.
- **httpd.conf** (Apache) contains the PHP module line LoadModule php_module and DocumentRoot.
- **Manual installation:** download PHP zip from php.net, extract to C:\php, rename php.ini-development to php.ini, add C:\php to the **PATH** variable, and configure Apache to load PHP.
- Restart Apache after every change in php.ini or httpd.conf.

Conclusion: Using XAMPP, PHP can be installed and running on Windows in a few minutes. php.ini is used to configure PHP's behaviour and extensions.

Q2. Explain different types of operators used in PHP with example

An **operator** is a symbol that performs an operation on one or more values (operands). PHP provides many types of operators for calculations, comparisons, logic and strings.

- **Arithmetic:** + (add), - (subtract), * (multiply), / (divide), % (modulus), ** (power). Example: $a = 10 % 3; // 1
- **Assignment:** =, +=, -=, *=, /=, .= Example: $x += 5; is the same as $x = $x + 5;
- **Comparison:** == (equal), === (identical: equal and same type), != or <>, !==, <, >, <=, >=, <=> (spaceship). Example: 5 == "5" is true, 5 === "5" is false.
- **Increment/Decrement:** ++$a (pre-increment), $a++ (post-increment), --$a, $a--.
- **Logical:** && or and, || or or, ! (not), xor. Example: if ($age > 18 && $age < 60)
- **String:** . (concatenation) and .= (concatenation assignment). Example: $name = "Hello " . "PHP";
- **Array:** + (union), ==, ===, != Example: $c = $a + $b;
- **Conditional (Ternary):** condition ? value1 : value2. Example: $result = ($marks >= 40) ? "Pass" : "Fail";
- **Null coalescing:** ?? returns the first value that is not null. Example: $user = $_GET['user'] ?? "Guest";
- **Bitwise:** & (AND), | (OR), ^ (XOR), ~ (NOT), << (left shift), >> (right shift).

Example

```
<?php
$a = 10; $b = 3;
echo $a + $b;            // 13
echo $a % $b;            // 1
echo ($a > $b) ? "a is big" : "b is big";   // a is big
echo "Sum = " . ($a + $b);                  // Sum = 13
?>
```

Conclusion: PHP operators are used to perform calculations, compare values, combine conditions and join strings. Choosing == or === correctly is important because PHP is loosely typed.

Q3. List and explain different loops used in PHP with its syntax and example.

A **loop** executes a block of code **repeatedly** while a condition is true. Loops reduce code length and are used for repeated tasks such as printing tables or reading arrays. PHP has four loops.

- **while loop:** Checks the condition **first**; the body runs only while it is true (entry-controlled).

```
while (condition) { statements; }

$i = 1;
while ($i <= 5) { echo $i . " "; $i++; }        // 1 2 3 4 5
```

- **do...while loop:** The body runs **at least once**, then the condition is checked (exit-controlled).

```
do { statements; } while (condition);

$i = 1;
do { echo $i . " "; $i++; } while ($i <= 5);    // 1 2 3 4 5
```

- **for loop:** Used when the number of iterations is known; initialization, condition and increment are written in one line.

```
for (init; condition; increment) { statements; }

for ($i = 1; $i <= 5; $i++) { echo $i . " "; }  // 1 2 3 4 5
```

- **foreach loop:** Used only with **arrays**; it takes each element one by one.

```
foreach ($array as $value) { ... }
foreach ($array as $key => $value) { ... }

$marks = array("Rahul" => 82, "Priya" => 90);
foreach ($marks as $name => $m) { echo "$name : $m <br>"; }
```

- **break** stops the loop immediately; **continue** skips the current iteration and goes to the next.

Conclusion: while and do-while are used when the number of repetitions is not fixed, for is used when it is known, and foreach is the simplest way to traverse arrays.

Q4. Write a code for embedding PHP in web pages

PHP code is **embedded inside HTML** pages. The web server executes the PHP part and sends only the resulting HTML to the browser. The file must be saved with the **.php** extension.

- **Standard tags:** <?php ... ?> — recommended and always enabled.
- **Short echo tag:** <?= expression ?> — prints a value; same as <?php echo expression; ?>
- **Short tags:** <? ... ?> — work only if short_open_tag = On in php.ini (not recommended).
- PHP can be placed **anywhere** in the HTML — in the head, body, attributes or inside loops that generate HTML.
- Each PHP statement ends with a **semicolon (;)**.

Program

```
<!DOCTYPE html>
<html>
<head>
    <title><?php echo "PHP Embedded Page"; ?></title>
</head>
<body>
    <h2>Welcome to PHP</h2>

    <?php
        $name = "Rahul";
        echo "<p>Hello, $name!</p>";
        echo "<p>Today is " . date("d-m-Y") . "</p>";
    ?>

    <p>Course: <?= "MCA" ?></p>

    <ul>
    <?php for ($i = 1; $i <= 3; $i++) { ?>
        <li>Item <?= $i ?></li>
    <?php } ?>
    </ul>
</body>
</html>
```

Output

```
Welcome to PHP
Hello, Rahul!
Today is 31-10-2026
Course: MCA
  • Item 1
  • Item 2
  • Item 3
```

Conclusion: Embedding PHP in HTML allows dynamic content (names, dates, database data) to be mixed with static HTML, which is the main strength of PHP for web development.

Q5. Define variable, write a note on scope of variables in PHP with example.

A **variable** is a named memory location used to store data. In PHP, a variable starts with the **$ sign** followed by its name (e.g. $name). PHP is **loosely typed** — the type is decided automatically by the value, and no declaration is needed.

- **Rules:** must start with $ followed by a letter or underscore; cannot start with a number; is case-sensitive ($age and $Age are different).
- **Scope** is the part of the program where a variable can be accessed. PHP has four scopes:
- **Local scope:** A variable declared **inside a function** can be used only inside that function.
- **Global scope:** A variable declared **outside all functions** cannot be used directly inside a function. It is accessed using the **global** keyword or the **$GLOBALS** array.
- **Static scope:** A **static** variable inside a function keeps its value between function calls instead of being destroyed.
- **Function parameters:** Arguments are local to the function they are passed to.

Example

```
<?php
$x = 10;                    // global variable

function test() {
    $y = 5;                 // local variable
    global $x;              // access global variable
    echo "x = $x, y = $y <br>";
}

function counter() {
    static $count = 0;      // static variable
    $count++;
    echo "Count = $count <br>";
}

test();
counter();
counter();
counter();
?>
```

Output

```
x = 10, y = 5
Count = 1
Count = 2
Count = 3
```

Conclusion: Understanding scope prevents errors such as "undefined variable" inside functions. Local variables keep functions independent, global gives shared access, and static remembers values between calls.

Q6. Write a PHP program to display the following output (half diamond pattern)

This pattern is a **half diamond**: the number of stars increases from 1 to 5 and then decreases from 4 to 1. It is printed using **nested for loops** — the outer loop controls the rows and the inner loop prints the stars.

- The first loop prints rows 1 to 5, with i stars in row i.
- The second loop prints rows 4 down to 1 for the lower half.
- **<pre>** is used so that the browser keeps the spaces and line breaks (\n).

Program

```
<?php
$n = 5;
echo "<pre>";
// upper half
for ($i = 1; $i <= $n; $i++) {
    for ($j = 1; $j <= $i; $j++) {
        echo "* ";
    }
    echo "\n";
}
// lower half
for ($i = $n - 1; $i >= 1; $i--) {
    for ($j = 1; $j <= $i; $j++) {
        echo "* ";
    }
    echo "\n";
}
echo "</pre>";
?>
```

Output

```
*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*
```

Conclusion: The pattern is produced by two pairs of nested loops — one increasing and one decreasing.

Q7. Write a PHP program to display the following output (pyramid pattern)

This is a **pyramid pattern** with 1, 3, 5, 7 and 9 stars in five rows, centred. Each row needs some **leading spaces** followed by an odd number of stars.

- Row i has **2i − 1 stars**.
- Row i has **(n − i)** blocks of two spaces before the stars, so the pyramid is centred.
- Nested loops: one inner loop for spaces and one inner loop for stars.

Program

```
<?php
$n = 5;
echo "<pre>";
for ($i = 1; $i <= $n; $i++) {
    for ($s = 1; $s <= $n - $i; $s++) {
        echo "  ";              // leading spaces
    }
    for ($j = 1; $j <= 2 * $i - 1; $j++) {
        echo "* ";
    }
    echo "\n";
}
echo "</pre>";
?>
```

Output

```
        *
      * * *
    * * * * *
  * * * * * * *
* * * * * * * * *
```

Conclusion: By printing decreasing spaces and an increasing odd number of stars in each row, a centred pyramid is formed.

Q8. Write a PHP program to display the following output (right-aligned half diamond)

This pattern is a **right-aligned half diamond**: stars increase from 1 to 5 and decrease back to 1, aligned to the right side. Leading spaces push the stars to the right.

- Row i has **i stars** and **(n − i)** blocks of two spaces before them.
- The first loop prints the upper half (1 to 5 stars), the second the lower half (4 to 1 stars).
- <pre> keeps the spacing in the browser.

Program

```
<?php
$n = 5;
echo "<pre>";
// upper half
for ($i = 1; $i <= $n; $i++) {
    for ($s = 1; $s <= $n - $i; $s++) echo "  ";
    for ($j = 1; $j <= $i; $j++) echo "* ";
    echo "\n";
}
// lower half
for ($i = $n - 1; $i >= 1; $i--) {
    for ($s = 1; $s <= $n - $i; $s++) echo "  ";
    for ($j = 1; $j <= $i; $j++) echo "* ";
    echo "\n";
}
echo "</pre>";
?>
```

Output

```
        *
      * *
    * * *
  * * * *
* * * * *
  * * * *
    * * *
      * *
        *
```

Conclusion: Adding leading spaces before the stars converts the left half diamond into a right-aligned half diamond.

Q9. Write a PHP program to display the following output (inverted pyramid)

This is an **inverted pyramid**: 9, 7, 5, 3 and 1 stars, centred. It is the reverse of the normal pyramid, so the outer loop runs from n down to 1.

- Row i (from 5 to 1) has **2i − 1 stars**.
- Leading spaces **(n − i)** blocks of two spaces increase as the rows go down.

Program

```
<?php
$n = 5;
echo "<pre>";
for ($i = $n; $i >= 1; $i--) {
    for ($s = 1; $s <= $n - $i; $s++) {
        echo "  ";              // leading spaces
    }
    for ($j = 1; $j <= 2 * $i - 1; $j++) {
        echo "* ";
    }
    echo "\n";
}
echo "</pre>";
?>
```

Output

```
* * * * * * * * *
  * * * * * * *
    * * * * *
      * * *
        *
```

Conclusion: Running the outer loop in reverse order with increasing spaces produces the inverted pyramid.

Q10. Explain in detail flow control statements used in PHP programming language.

Flow control statements decide the **order in which statements are executed**. By default statements run one after another; flow control lets a program take decisions, repeat code or jump out of a block.

Conditional (Decision) Statements

- **if:** executes a block only when the condition is true. if ($age >= 18) { echo "Adult"; }
- **if...else:** one block when true, another when false.
- **if...elseif...else:** checks many conditions one by one.

```
if ($marks >= 75) echo "Distinction";
elseif ($marks >= 60) echo "First Class";
else echo "Pass";
```

- **switch:** selects one block out of many based on the value of an expression; break stops falling into the next case.

```
switch ($day) {
    case 1: echo "Monday"; break;
    case 2: echo "Tuesday"; break;
    default: echo "Other day";
}
```

- **match (PHP 8):** like switch but returns a value and uses strict comparison.

Looping Statements

- **while, do...while, for, foreach** — repeat a block of code (see loops).

Jump Statements

- **break:** exits a loop or switch immediately.
- **continue:** skips the rest of the current iteration.
- **return:** ends a function and sends back a value.
- **exit / die():** stops the whole script, e.g. die("Connection failed");
- **include / require:** transfer control to code in another file.

Conclusion: Flow control statements give logic to a PHP program — conditions choose what to do, loops repeat work and jump statements change the normal flow.

Q11. Explain Different datatypes used in PHP.

A **datatype** specifies the type of value a variable holds. PHP is **loosely (dynamically) typed** — the datatype is decided automatically from the assigned value. PHP supports **eight datatypes** in three groups.

Scalar (single value) types

- **Integer:** whole numbers without a decimal point, positive or negative. $age = 25;
- **Float (double):** numbers with a decimal point or in exponent form. $price = 99.50;
- **String:** a sequence of characters in single or double quotes. $name = "Rahul"; Double quotes expand variables, single quotes do not.
- **Boolean:** only two values, **true** or **false**. $isPass = true;

Compound types

- **Array:** stores multiple values in one variable — indexed, associative or multidimensional. $colors = array("Red", "Green");
- **Object:** an instance of a class that holds properties and methods. $s = new Student();

Special types

- **NULL:** a variable with no value. $x = null;
- **Resource:** a reference to an external resource such as a database connection or file handle. $f = fopen("a.txt", "r");

Checking the type

- **gettype($x)** returns the type name; **var_dump($x)** shows type and value; is_int(), is_string(), is_array() check a type; settype() and (int) casting change it.

```
<?php
$a = 10;  $b = 2.5;  $c = "PHP";  $d = true;
var_dump($a, $b, $c, $d);
// int(10) float(2.5) string(3) "PHP" bool(true)
?>
```

Conclusion: PHP has four scalar, two compound and two special datatypes. Because PHP converts types automatically, functions like var_dump() and === help avoid type-related errors.

Q12. Write a PHP program, to find out factorial using function.

The **factorial** of a number n (n!) is the product of all positive integers from 1 to n, e.g. 5! = 5 × 4 × 3 × 2 × 1 = 120, and 0! = 1. A **user-defined function** makes the code reusable — it takes the number as a parameter and returns its factorial.

- Define a function **factorial($n)** using the **function** keyword.
- Use a loop to multiply numbers from 1 to n (iterative method), or call the function itself (recursive method).
- Call the function with a number and display the returned value.

Program

```
<?php
// Iterative method
function factorial($n) {
    $fact = 1;
    for ($i = 1; $i <= $n; $i++) {
        $fact = $fact * $i;
    }
    return $fact;
}

// Recursive method
function factRec($n) {
    if ($n <= 1)
        return 1;
    return $n * factRec($n - 1);
}

$num = 5;
echo "Factorial of $num (loop) = " . factorial($num) . "<br>";
echo "Factorial of $num (recursion) = " . factRec($num);
?>
```

Output

```
Factorial of 5 (loop) = 120
Factorial of 5 (recursion) = 120
```

Conclusion: Using a function makes the factorial logic reusable. Both the loop and recursive methods give the same result.

Q13. Explain the terms: a) POSIX-Style Regular Expressions b) Perl-Compatible Regular Expressions.

A **regular expression (regex)** is a pattern used to **search, match, validate and replace text**, e.g. checking an email address or phone number. PHP has supported two types of regular expressions: **POSIX** and **Perl-Compatible (PCRE)**.

a) POSIX-Style Regular Expressions

- Based on the POSIX standard; patterns are written **without delimiters**.
- Functions: **ereg()** (case-sensitive match), **eregi()** (case-insensitive), **ereg_replace()**, **eregi_replace()**, **split()**.
- Use **character classes** such as [[:alpha:]], [[:digit:]], [[:space:]], [[:upper:]].
- Example: ereg("^[0-9]{10}$", $mobile) checks a 10-digit number.
- **Deprecated in PHP 5.3 and removed in PHP 7**, so they should not be used in new code.

b) Perl-Compatible Regular Expressions (PCRE)

- Based on Perl syntax; patterns are written **inside delimiters**, usually /pattern/.
- **Modifiers** after the delimiter: i (case-insensitive), m (multiline), s, x, u (Unicode).
- Functions: **preg_match()**, **preg_match_all()**, **preg_replace()**, **preg_split()**, **preg_quote()**, **preg_grep()**.
- Support shorthand classes \d (digit), \w (word character), \s (space), and anchors ^ and $.
- Faster and more powerful; the **standard in modern PHP**.

```
<?php
$email = "rahul@gmail.com";
if (preg_match("/^[\w.]+@[\w]+\.[a-z]{2,}$/i", $email))
    echo "Valid email";            // Valid email

echo preg_replace("/\s+/", " ", "PHP    is   fun");   // PHP is fun
?>
```

Conclusion: POSIX regex functions (ereg) are old and removed from PHP 7, while PCRE functions (preg_*) are faster, more powerful and are used today for pattern matching in PHP.

Q14. What is session? Where you will use it in web programming? Explain use of session start.

HTTP is a **stateless** protocol — it does not remember a user from one page to the next. A **session** stores user information **on the server** across multiple pages. Each user gets a unique **session ID**, which is stored in the browser as a cookie (PHPSESSID).

Where Sessions are Used

- **Login systems:** to remember that a user is logged in on every page.
- **Shopping carts:** to keep selected items while the user browses.
- **User preferences:** language, theme, etc. during a visit.
- **Multi-page forms** and keeping the user's name or role for access control.

Use of session_start()

- **session_start()** starts a new session or **resumes the existing session** of the user.
- It must be called **at the top of every page** that uses session data, **before any HTML output**.
- After calling it, data is stored and read through the **$_SESSION** superglobal array.
- **session_unset()** removes all session variables; **session_destroy()** destroys the session (logout).

```
<?php
session_start();                        // start / resume session
$_SESSION['user'] = "Rahul";            // store data
echo "Welcome " . $_SESSION['user'];    // read data on any page

// logout
// session_unset();
// session_destroy();
?>
```

Output

```
Welcome Rahul
```

Conclusion: Sessions let a web application remember users across pages securely on the server. session_start() must be called first on every page that uses $_SESSION.

Q15. What are cookies? Explain how cookies are set, View and deleted?

A **cookie** is a small piece of data (name–value pair, max about 4 KB) that the server sends to the browser, which **stores it on the client's computer**. The browser sends the cookie back with every request to the same website, so the site can remember the user.

- Used for: "remember me" logins, user preferences, tracking visits and shopping carts.
- **Setting a cookie:** use **setcookie(name, value, expire, path, domain, secure, httponly)**. It must be called **before any HTML output**. The expiry time is in seconds from now, e.g. time() + 3600 for 1 hour.
- **Viewing a cookie:** cookies are read using the **$_COOKIE** superglobal array. A new cookie is available only on the **next page load**. Use isset() to check whether it exists.
- **Deleting a cookie:** call setcookie() with the same name and an **expiry time in the past**, e.g. time() - 3600; the browser then removes it.
- **Session cookie:** if no expiry is given, the cookie is deleted when the browser closes.

```
<?php
// Set cookie for 1 hour
setcookie("user", "Rahul", time() + 3600);

// View cookie
if (isset($_COOKIE["user"]))
    echo "User is: " . $_COOKIE["user"];
else
    echo "Cookie not set";

// Delete cookie
// setcookie("user", "", time() - 3600);
?>
```

Output (after refreshing the page)

```
User is: Rahul
```

Conclusion: Cookies store small data on the client side. They are set with setcookie(), read with $_COOKIE and deleted by setting a past expiry time. Sensitive data should be kept in sessions, not cookies.

Q16. Write a program to connect PHP with MySQL.

PHP connects to a MySQL database using the **MySQLi** extension (or PDO). Once connected, the program can create tables, insert, update, delete and select data using SQL queries.

- **mysqli_connect(host, username, password, database)** opens the connection; it returns false on failure.
- **mysqli_connect_error()** returns the error message if the connection fails.
- **mysqli_query($conn, $sql)** executes an SQL query.
- **mysqli_fetch_assoc($result)** reads one row at a time as an associative array.
- **mysqli_close($conn)** closes the connection.
- In XAMPP the default user is **root** with an **empty password**.

Program

```
<?php
$host = "localhost";
$user = "root";
$pass = "";
$db   = "college";

// Connect to MySQL
$conn = mysqli_connect($host, $user, $pass, $db);

if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
echo "Connected successfully<br>";

// Fetch data
$result = mysqli_query($conn, "SELECT id, name FROM student");
while ($row = mysqli_fetch_assoc($result)) {
    echo $row['id'] . " - " . $row['name'] . "<br>";
}

mysqli_close($conn);
?>
```

Output

```
Connected successfully
1 - Rahul
2 - Priya
3 - Amit
```

Conclusion: With mysqli_connect() and mysqli_query(), PHP can easily work with MySQL, which makes it ideal for building database-driven web applications.

Q17. Explain Basic drawing functions with example.

PHP can create images dynamically using the **GD library** (extension=gd in php.ini). Images such as charts, graphs, CAPTCHA and buttons are drawn on a canvas using drawing functions and sent to the browser as PNG, JPEG or GIF.

- **imagecreate(width, height)** / **imagecreatetruecolor()** — creates a blank image (canvas).
- **imagecolorallocate($img, r, g, b)** — creates a colour; the first allocated colour becomes the background in imagecreate().
- **imageline($img, x1, y1, x2, y2, color)** — draws a line.
- **imagerectangle()** / **imagefilledrectangle($img, x1, y1, x2, y2, color)** — draws an outlined / filled rectangle.
- **imageellipse()** / **imagefilledellipse($img, cx, cy, w, h, color)** — draws a circle or ellipse.
- **imagearc()**, **imagepolygon()** — draw arcs and polygons.
- **imagestring($img, font, x, y, text, color)** — writes text on the image.
- **header("Content-Type: image/png")** and **imagepng($img)** — send the image to the browser.
- **imagedestroy($img)** — frees the memory.

Example

```
<?php
$img   = imagecreate(300, 200);
$white = imagecolorallocate($img, 255, 255, 255);   // background
$red   = imagecolorallocate($img, 255, 0, 0);
$blue  = imagecolorallocate($img, 0, 0, 255);
$black = imagecolorallocate($img, 0, 0, 0);

imageline($img, 10, 10, 290, 10, $black);
imagerectangle($img, 20, 30, 120, 100, $blue);
imagefilledellipse($img, 200, 70, 80, 80, $red);
imagestring($img, 5, 80, 150, "PHP GD Drawing", $black);

header("Content-Type: image/png");
imagepng($img);
imagedestroy($img);
?>
```

Output: A 300×200 white image with a black line, a blue rectangle, a red filled circle and the text "PHP GD Drawing".

Conclusion: GD drawing functions allow PHP to generate images on the fly, which is useful for graphs, CAPTCHA, thumbnails and dynamic buttons.

Q18. Write note on dynamically generated buttons.

Dynamically generated buttons are button images that are **created by PHP at run time** using the GD library, instead of making a separate image file for every button in an image editor. The button text is passed in the URL, and the same script creates any number of buttons.

- **Working:** the HTML page uses an <img> tag whose src is a PHP script, e.g. <img src="button.php?text=Home">.
- The script reads the text from **$_GET**, loads a **blank button template** with imagecreatefrompng("button.png"), writes the text with **imagettftext()** or **imagestring()**, and outputs the image.
- **Centring text:** imagettfbbox() gives the size of the text, which is used to calculate the x and y position.
- **Caching:** generating the image on every request is slow, so the script saves the button as a file the first time and sends the saved file next time — this improves performance.
- **Advantages:** consistent look, buttons in any language/text, easy to change the design in one place, less manual work.
- **Disadvantages:** uses server CPU; needs the GD extension and fonts on the server.

```
<?php
// button.php?text=Home
$text = isset($_GET['text']) ? $_GET['text'] : "Button";
$img  = imagecreatefrompng("button.png");          // blank template
$black = imagecolorallocate($img, 0, 0, 0);
$x = (imagesx($img) - imagefontwidth(5) * strlen($text)) / 2;
$y = (imagesy($img) - imagefontheight(5)) / 2;
imagestring($img, 5, $x, $y, $text, $black);
header("Content-Type: image/png");
imagepng($img);
imagedestroy($img);
?>

<!-- usage in HTML -->
<img src="button.php?text=Home">  <img src="button.php?text=Contact">
```

Conclusion: Dynamically generated buttons save design effort by creating button images from one PHP script; caching makes them fast enough for real websites.

Q19. Explain procedure to create PDF in PHP and write example

PHP can generate **PDF documents** dynamically, for example invoices, mark sheets and reports. This is done using the **PDFlib extension** or a pure-PHP library such as **FPDF** (free, no extension needed) or TCPDF.

Procedure (using FPDF)

- **Step 1:** Download FPDF from fpdf.org and copy fpdf.php (and the font folder) into the project.
- **Step 2:** Include the library — require("fpdf.php");
- **Step 3:** Create a PDF object — $pdf = new FPDF();
- **Step 4:** Add a page — $pdf->AddPage(); (orientation and size can be given, e.g. 'P', 'A4').
- **Step 5:** Set the font — $pdf->SetFont('Arial', 'B', 16);
- **Step 6:** Add content — **Cell()** for a line/box of text, **MultiCell()** for paragraphs, **Ln()** for a line break, **Image()** for pictures, **Line()** and **Rect()** for graphics.
- **Step 7:** Output the document — $pdf->Output(); sends it to the browser; Output('F', 'file.pdf') saves it on the server; 'D' forces download.
- With **PDFlib**: pdf_new(), pdf_begin_document(), pdf_begin_page_ext(), pdf_setfont(), pdf_show_xy(), pdf_end_page_ext(), pdf_end_document().

Example

```
<?php
require("fpdf.php");

$pdf = new FPDF();
$pdf->AddPage();
$pdf->SetFont('Arial', 'B', 16);
$pdf->Cell(0, 10, 'Student Report', 0, 1, 'C');

$pdf->SetFont('Arial', '', 12);
$pdf->Cell(40, 10, 'Name: Rahul');
$pdf->Ln();
$pdf->Cell(40, 10, 'Course: MCA');
$pdf->Ln();
$pdf->Cell(40, 10, 'Marks: 82');

$pdf->Output();            // display in browser
?>
```

Output: The browser opens a PDF page with the heading "Student Report" centred, followed by Name, Course and Marks.

Conclusion: With libraries like FPDF, a PDF is created in a few steps — create object, add page, set font, add content and output — so PHP applications can produce printable reports easily.

Q20. Explain Introspection method in brief.

Introspection is the ability of a program to **examine its own classes, objects, properties and methods at run time** — for example finding the class of an object or listing its methods. It is useful for writing generic code, debugging and frameworks.

- **class_exists("Name")** — checks whether a class is defined.
- **get_class($obj)** — returns the class name of an object.
- **get_parent_class($obj)** — returns the name of the parent class.
- **get_class_methods("Name")** — returns an array of the class's method names.
- **get_class_vars("Name")** — returns the default properties of a class.
- **get_object_vars($obj)** — returns the properties and current values of an object.
- **method_exists($obj, "name")** / **property_exists($obj, "name")** — check whether a method or property exists.
- **is_a($obj, "Name")** / **is_subclass_of($obj, "Name")** — check the class relationship.
- **get_declared_classes()** — lists all defined classes.

```
<?php
class Person { public $name = "Rahul"; function show() {} }
class Student extends Person { public $course = "MCA"; function study() {} }

$s = new Student();
echo get_class($s) . "<br>";                    // Student
echo get_parent_class($s) . "<br>";             // Person
print_r(get_class_methods("Student"));          // Array ( [0] => study [1] => show
)
print_r(get_object_vars($s));                   // Array ( [name] => Rahul [course]
=> MCA )
echo method_exists($s, "study") ? "Yes" : "No";  // Yes
?>
```

Conclusion: Introspection functions let PHP inspect classes and objects dynamically, which helps in debugging, serialization and building flexible, reusable code.